BlumNet: Graph Component Detection for Object Skeleton Extraction

Yulu Zhang*
Artificial Intelligence Group
Huipainter
Shanghai, China

Liang Sang
Computer Vision Group
Transsion
Shanghai, China

Marcin Grzegorzek
University of Lübeck, Lübeck, Germany. University of Economics in Katowice, Katowice, Poland

John See
School of Mathematical and Computer Sciences, Heriot-Watt University, Malaysia

Cong Yang†
School of Future Science and Engineering, Soochow University
Suzhou, China

ABSTRACT

In this paper, we present a simple yet efficient framework, BlumNet, for extracting object skeletons in natural images and binary shapes. With the need for highly reliable skeletons in various multimedia applications, the proposed BlumNet is distinguished in three aspects: (1) The inception of graph decomposition and reconstruction strategies further simplifies the skeleton extraction task into a graph component detection problem, which significantly improves the accuracy and robustness of extracted skeletons. (2) The intuitive representation of each skeleton branch with multiple structured and overlapping line segments can effectively prevent the skeleton branch vanishing problem. (3) In comparison to traditional skeleton heatmaps, our approach directly outputs skeleton graphs, which is more feasible for real-world applications. Through comprehensive experiments, we demonstrate the advantages of BlumNet: significantly higher accuracy than the state-of-the-art AdaLSN (0.826 vs. 0.786) on the SK1491 dataset, a marked improvement in robustness on mixed object deformations, and also a state-of-the-art performance on binary shape datasets (e.g. 0.893 on the MPEG7 dataset).

CCS CONCEPTS

• **Computing methodologies** → **Shape representations**.

KEYWORDS

Skeleton, Skeleton Graph, Skeleton Extraction, Transformer

ACM Reference Format:
Yulu Zhang, Liang Sang, Marcin Grzegorzek, John See, and Cong Yang. 2022. BlumNet: Graph Component Detection for Object Skeleton Extraction. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22), October 10–14, 2022, Lisboa, Portugal.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3503161.3547816

*Equal Contribution: Yulu Zhang and Cong Yang.
†Corresponding Author: Cong Yang (yangcong955@126.com)

MM '22, October 10–14, 2022, Lisboa, Portugal

ACM ISBN 978-1-4503-9203-7/22/10…$15.00
https://doi.org/10.1145/3503161.3547816

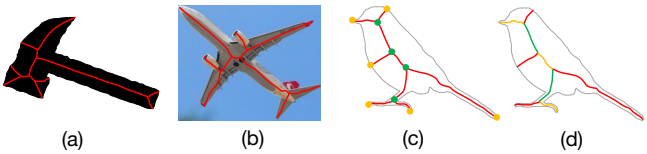

Figure 1: Skeleton graphs: (a) in binary shape, (b) in natural image, (c) endpoints (orange) and junction points (green), (d) skeleton branches with different colours.

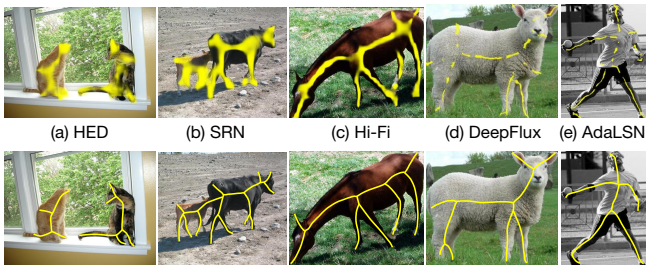

Figure 2: Comparison of image skeleton extraction: (a) HED [55], (b) SRN [19], (c) Hi-Fi [70], (d) DeepFlux [53], (e) AdaLSN [29], and our proposed (f) BlumNet.

1 INTRODUCTION

Skeletons (or medial axis) have the potential to provide a compact but meaningful object representation in binary shapes (Fig. 1 (a)) and natural images (Fig. 1 (b)) (hereafter referred to them as "shape" and "image", respectively), which are suitable for image representation [1] and various multimedia applications [30, 47, 50, 54, 69]. In practice, an object skeleton is normally encoded in the graph format, namely "skeleton graph", for the ease of skeleton pruning [4], matching [3], classification [5], and analysis [8, 25, 39, 64] tasks. For clarity in terminology, various skeleton graph components are commonly defined as follows (see Fig. 1 (c),(d)): 1) Endpoint: a skeleton point has only one point adjacent. 2) Junction point: has three or more points adjacent. 3) Connection point: neither endpoint nor junction point. 4) Skeleton branch: the sequence of connection points within two directly connected skeleton points.

1.1 Challenges on Skeleton Extraction

While existing methods mainly focused on skeleton extraction using Convolutional Neural Networks (CNN) [29, 33, 45], it is still hard to apply them in practice, particularly in scenarios where high-quality skeleton graphs are required [8, 25, 39]. This is because

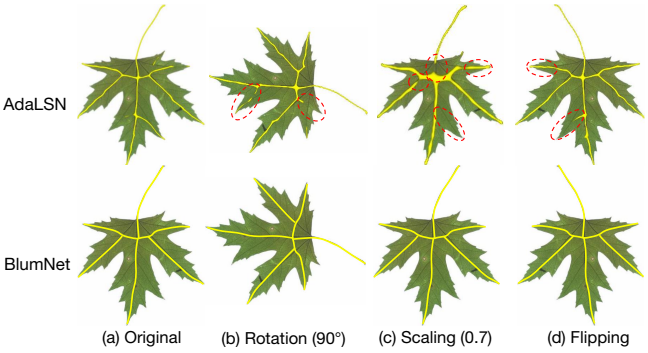

Figure 3: Evaluation of AdaLSN [29] (top) and our proposed BlumNet (bottom) with respect to three image deformations. Skeletonal differences are marked with red ellipses.

one crucial restriction in existing methods lies in their limited *quality* and *robustness* of extracted skeletons with respect to graph completeness and stability, respectively.

Limited quality: Most methods either cannot guarantee the topological and geometrical features in the representation, or they output noisy, disjointed and incomplete skeleton branches in heat maps (also called skeleton maps). For instance, skeletons from HED (Holistically-Nested Edge Detection) [55], SRN (Side-output Residual Network) [19] and Hi-Fi (Hierarchical Feature integration) [70] contained lots of noise and limited smoothness (see Fig. 2). DeepFlux [53] and AdaLSN (Adaptive Linear Span Network) [29] output clearer and slimmer results, while there remains some false positive points, disjointed segments and incomplete branches. These methods are not feasible to generate high-quality skeleton graphs.

Limited robustness: Existing methods are not very robust towards deformations from objects and images. Take the state-of-the-art method AdaLSN for example, the extracted skeletons in rotated, flipped and re-scaled images are all different from the original one (see Fig. 3), although the training data was augmented accordingly. The main reason is that existing methods extract skeletons purely based on high-level semantics from deep layers of CNNs [29]. In other words, these methods mainly focus on the global skeleton structure rather than fine-grained skeleton branches. To verify this, observe carefully the red ellipses in Fig. 3: any deformations in geometry easily lead to the vanishing, twisting and extension of fine-grained segments. As a result, the generated skeleton graphs are unstable, especially at its branches.

1.2 Tackling the Challenges

In light of these challenges, we introduce a novel framework, BlumNet, for object skeleton extraction from images. BlumNet is named after Harry Blum[1], who was a pioneer in the definition and development of shape skeletons [6]. We design BlumNet to properly tackle the quality and robustness challenges as it properly incorporates both coarse- (global skeleton structure) and fine-grained (local branch distribution) features for skeleton graph extraction. In brief, BlumNet decomposes a skeleton graph into structured components and simplifies the skeleton extraction problem into graph component detection and assembling tasks.

[1]Harry Blum (1924-1987) was a scientist at National Institutes of Health, Bethesda, MD. known for his work in topological skeletons.

Specifically, BlumNet comprises of three parts: (a) A skeleton graph decomposition strategy for training data preparation, (b) a Graph Component Detector (GCD) for initial component detection and identification, and (c) a Graph Reconstructor (GR) for skeleton branch and final skeleton graph reconstruction. We also increase the effectiveness of BlumNet by further decomposing skeleton branches into overlapping line segments. This strategy can effectively prevent branch vanishing by detecting fine-grained and overlapping line segments. It also improves the efficiency of GCD by simplifying its tasks– encoding points (end- and junction points) and line segments with simple vectors. As such, existing line segment detectors [58–60, 68, 71] can be extended for the GCD tasks. For instance, we can incorporate the transformer architecture [72] into the GCD to capture spatial and context features for point and line segment detection (see Section 3). Since line segments from branches, noises and objects are easily confused, two additional metrics (isBranch and Graph ID) are jointly generated with each line vector to identify its group assignment on skeletons and graphs. As a result, the GR is inherently more noise-insensitive and supports multiple objects.

The rationale behind BlumNet is that it extracts skeletons via a decomposition-detection-reconstruction process whereby both global and local skeleton graph features are effectively involved. Consequently, for the *limited quality* challenge, the quality of extracted skeletons are dramatically improved, as shown in Fig. 2 (f). This is because, different from existing methods require heavy and semi-automatic processing on heat maps, BlumNet directly outputs slim, low-noise and identified skeleton graphs. For the *limited robustness* challenge, since BlumNet encodes object context via global and local graph features, it is insensitive to object rotation, rescaling, and flipping. Fig. 3 (bottom) shows evidence of Blumnet's much more robust performance against shape deformations.

1.3 Contributions

Succinctly, the main contributions of this work are as follows:1) We propose a simple yet efficient framework, BlumNet, for accurate and robust object skeleton extraction in images and shapes. BlumNet encodes both global and local graph features by simplifying the skeleton extraction problem into a graph component detection and reconstruction tasks. BlumNet is highly suitable for real-world multimedia applications as it supports multiple objects, yielding slim and precise skeleton graphs. 2) We provide an extensive set of comparison experiments and ablation studies to demonstrate the effectiveness of BlumNet. We particularly show superior performances against state-of-the-art methods (0.826 vs. 0.786 for AdaLSN on the SK1491 dataset), a marked improvement in robustness against mixed object deformations, and also state-of-the-art accuracy on actively used shape datasets.

2 RELATED WORKS

We present a concise survey of existing skeleton extraction methods in images and shapes. We have organized them into two groups, focusing respectively on traditional and CNN-based methods.

2.1 Traditional Methods

Early skeletonization methods were commonly performed on shapes via morphological thinning [67] and geometrical operations based on Voronoi diagrams [7], distance transformations [2, 10, 13, 20]

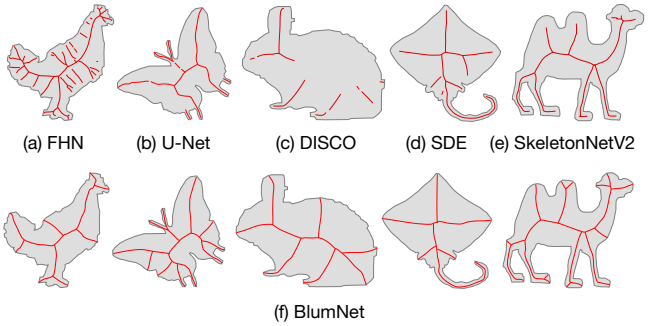

Figure 4: Shape skeleton extraction using CNN-based methods: FHN [18], U-Net [33], DISCO [48], SDE [49], SkeletonNetV2 [33] and our proposed BlumNet.

and curve evolution [4, 63], etc. Recent research works mainly focus on automation [43, 61] and application [38] of these methods. However, they can only be applied on segmented foreground and binary targets like characters. They cannot compute skeletons directly from images. For this, some unsupervised methods [16, 27, 66] were introduced based on gradient intensity map, driven by geometric constraints between skeleton points and edge fragments. While such methods can only work on simple images with prior object shape and location, later works [24, 42, 46, 47, 51, 52] tended to use learning-based approaches to improve the skeletonization.

For instance, Levinshtein *et al.* employ multi-scale super-pixels and a learned affinity between adjacent super-pixels to group proximal skeleton points [24]. This method was later optimized by Lee *et al.* with a deformable disc model to detect curved and tapered symmetric parts [46]. Tsogkas *et al.* introduce a hand-designed descriptor for each pixel and then train a classifier for symmetry detection [52]. This method was also extended by Shen *et al.* to improve the diversity of symmetry patterns [42]. However, these methods are still unable to cope with complex backgrounds or clutter in images. Recently, Jerripothula *et al.* introduce a coupled framework for joint co-skeletonization and co-segmentation so that they benefit each other synergistically [17]. However, this method still requires prior object shape and location information for accurate skeleton graph extraction.

Different from the above approaches, our proposed BlumNet is more generalized and has robust performance on a large variety of images (see Fig. 2 (f)). Moreover, it directly extracts skeleton graphs from multiple objects in an end-to-end fashion, which makes it more suitable for real-world multimedia applications.

2.2 CNN-based Methods

In recent years, CNN-based methods have had a tremendous impact on object skeleton extraction in shapes and images. Particularly, the annual SkelNetOn Workshop (on Pixel and Image tracks) has exhibited a rapid development of CNN-based skeleton extraction methods towards high accuracy and robustness [15].

For shapes, most existing methods employ a multi-scale architecture to encode shape features for skeleton regression. For instance, Jiang *et al.* introduced a Feature Hourglass Network (FHN), making use of feature pyramids for multi-scale feature extraction from shapes [18]. Nathan *et al.* introduce an encoder-decoder network named SkeletonNetV2, to leverage the benefits of the coordinated convolutional layer and multi-level supervision [33]. Nguyen *et al.* extended the U-Net architecture with attention mechanism to improve the multi-scale feature representation and reconstruction capabilities [35]. Similarly, Song *et al.* propose a DISCO (Dual Input Streams autoenCOder) architecture [48] and employed U-Net [37], U-ResNet [12], and U-Transformer [36] as backbone networks for skeletonization. Tang *et al.* pre-processed the input shape using Smooth Distance Estimation (SDE) and edge transformation, and finally regress the shape skeleton using a modified U-Net model and multi-model ensemble [49]. Recently, Ko *et al.* proposed an unconventional method based on deep adversarial network for font character skeletonization [21]. However, as presented in Fig. 4, a majority of these extracted skeletons either contain noisy and disjointed segments, or incomplete skeleton branches. This could cause complications towards usage in real-world applications.

For images, object skeleton extraction has been formulated as a pixel-wise binary classification problem with multi-scale feature integration [19, 28, 29, 53, 55, 70] whereby confidence scores are used to build skeleton heat maps. For instance, HED [55] utilize parallel multiple side-outputs to produce edge and skeleton heat maps based on learned edges and scale-specific skeleton representations. Meanwhile, SRN [19] builds short connections between adjacent side-outputs in a deep-to-shallow manner for complementary feature utilization. This way, SRN can progressively expand the feature space to fit the errors between the ground-truth and the multiple side-outputs. Hi-Fi [70] hierarchically integrates multi-scale features with bidirectional guidance to capture high-level semantics from deeper layers as well as low-level details from shallower layers. DeepFlux [53] and GeoSkelNet [56] regress skeleton heat maps via a flux-based fashion. Particularly, they predict a region-based vector field thereby encoding the skeleton point positions to contextually meaningful entities. Different from these hand-designed networks, the architecture of AdaLSN is optimized based on a genetic architecture search [29]. However, as discussed in Section 1, these methods are still unable to produce high quality outputs for accurate and robust skeleton graph extraction (see Fig. 2 and 3).

Our proposed BlumNet is fundamentally different from existing methods, as it not only encodes both fine- and coarse skeleton features in the GCD, but also involves both local and global graph constraints in the GR to form the final skeleton. As a result, BlumNet effectively improves the quality and robustness of extracted skeletons in both shapes and images (see Fig. 2, 3 and 4).

3 BLUMNET

For an image $\mathbf{I}$, the ground truth skeleton graph $\mathbf{G}$ is composed by endpoints $\mathbf{E}$, junction points $\mathbf{J}$ and their corresponding skeleton branches $\mathbf{B}$:

$$\mathbf{G} = [\mathbf{E}, \mathbf{J}, \mathbf{B}] \quad . \tag{1}$$

Succinctly, we denote a single endpoint, junction point and branch in $\mathbf{G}$ as $\mathbf{e}$, $\mathbf{j}$ and $\mathbf{b}$, respectively. For images with multiple objects, we treat each object as a sub-graph in $\mathbf{G}$ and identify them by unique indices. Here we define the simplest graph $\mathbf{G}$ as a curve: a skeleton branch $\mathbf{b}$ with two endpoints $\mathbf{e}_1$ and $\mathbf{e}_2$, where $\mathbf{b} \in \mathbf{B}$, $\mathbf{e}_1, \mathbf{e}_2 \in \mathbf{E}$. We note that in SYMMAX300 [52] and SymPASCAL [19] datasets, the ground truth of some images is only a curve. With these notations, we now discuss details of the three parts of BlumNet.

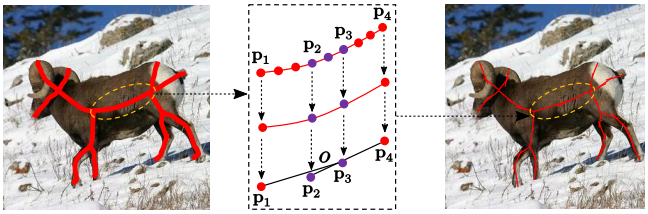

Figure 5: General idea of skeleton graph decomposition.

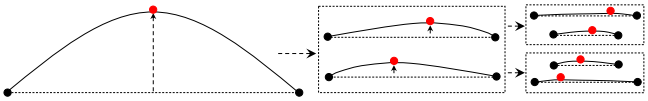

Figure 6: Searching the appropriate length for all $\mathbf{l}_k$.

3.1 Skeleton Graph Decomposition

For a ground truth skeleton $\mathbf{G}$, the main purpose of this part is to decompose it into point and line segments for GCD training. We formalize $\mathbf{G}$ by:

$$\mathbf{G} \approx [\mathbf{E}, \mathbf{J}, \mathbf{L}] \quad . \tag{2}$$

where $\mathbf{L}$ denotes the line segments extracted from $\mathbf{B}$. Let $\mathbf{l}_k$ denote a line segment in $\mathbf{L}$, $\mathbf{l}_k \in \mathbf{L}$ and $k \geq 1$. The maximum value of k (denoted by K) depends on the length of $\mathbf{l}_k$ ($|\mathbf{l}_k|$) and the degree of overlap $o \in [0, 1]$ between adjacent segments $(\mathbf{l}_k, \mathbf{l}_{k+1})$. An intuitive explanation is shown in Fig. 5 where $\mathbf{p}_1$ to $\mathbf{p}_4$ are four skeleton points along a branch while $\mathbf{p}_2$ and $\mathbf{p}_3$ anchor the overlapping. The branch is then approximated by the overlapped vectors $\overrightarrow{\mathbf{p}_1\mathbf{p}_3}$ and $\overrightarrow{\mathbf{p}_2\mathbf{p}_4}$, corresponding to the adjacent line segments $\mathbf{l}_k$ and $\mathbf{l}_{k+1}$. Accordingly, K is reduced (to less lines) if $|\mathbf{l}_k|$ is larger or o is smaller.

To improve the performance of GCD, all segments $\mathbf{l}_k$ are set to the same length in a dataset. This is because the detector is more focused on the location and direction of $\mathbf{l}_k$, if $|\mathbf{l}_k|$ is fixed. Here, we introduce a heuristic method to search for the appropriate $|\mathbf{l}_k|$ with the following steps: (1) Normalization: All ground truth images (containing $\mathbf{G}$) within a dataset is resized to ensure the larger side is 512 pixels. All $\mathbf{G}$ have a 1-pixel width. (2) Partitioning: As shown in Fig. 6, a curve is iteratively divided into sub-curves based on the vertices (the red points) with the largest distance to its endpoint connecting line, until the largest distance is less than 1 pixel. This way, the sub-curves are close to being straight lines. This partitioning method is applied to all skeleton branches and finally a curve "book" containing the divided curve segments is collected. (3) Statistics: The median value of all curve lengths within the book is used to determine $|\mathbf{l}_k|$. For instance, $|\mathbf{l}_k| = 11$ is computed for the SK1491 dataset [44]. The rationale behind this method is to properly balance the degree of decomposition and the preservation.

In practice, $\mathbf{l}_k$ is extracted by a sliding window with the fixed $|\mathbf{l}_k|$ along the path of $\mathbf{b}$ in $\mathbf{B}$ (e.g. $\overrightarrow{\mathbf{p}_1\mathbf{p}_3}$ in Fig. 5 (b)). After that, o is used to locate the starting point (e.g. $\mathbf{p}_2$) of the adjacent line segment $\mathbf{l}_{k+1}$ in clockwise order. For branches with shorter lengths than $|\mathbf{l}_k|$, we represent it by using $\mathbf{l}_k$ twice but in opposing directions, e.g. $(\overrightarrow{\mathbf{p}_1\mathbf{p}_3}, \overrightarrow{\mathbf{p}_3\mathbf{p}_1})$. In our experiments, o is empirically set to 0.6 based on the experiments in Fig. 11 as the overlapping and approximating strategy can effectively improve the accuracy of GCD. This is intuitive since it reduces the possibility of branch vanishing by detecting multiple, simple line segments to characterize complex skeletons. Our experiments in Sec. 4.2.1 verifies the effectiveness of this overlap strategy. Finally, as presented in Eq. 2, training data from $\mathbf{G}$ includes: (1) endpoints $\mathbf{E}$, (2) junction points $\mathbf{J}$, (3) Line segments $\mathbf{L}$, and (4) their sub-graph identification.

3.2 Graph Component Detection

Given an image $\mathbf{I}$, our proposed GCD predicts initial skeleton components using a single network $f(\mathbf{I}; \theta) = (\widehat{\mathbf{E}}, \widehat{\mathbf{J}}, \widehat{\mathbf{L}})$, where θ is the network weights, $\widehat{\mathbf{E}}$, $\widehat{\mathbf{J}}$ and $\widehat{\mathbf{L}}$ are predicted endpoints, junction points and line segments, respectively. $\widehat{\mathbf{E}}$ and $\widehat{\mathbf{J}}$ are distinguished by the point type. For the ease of Graph Reconstructor (GR), each line in $\widehat{\mathbf{L}}$ contains a confidence score (isBranch) representing the probability of a line belonging to $\mathbf{G}$. All elements in $\widehat{\mathbf{E}}$, $\widehat{\mathbf{J}}$ and $\widehat{\mathbf{L}}$ contain a Graph ID which identifies different objects within $\mathbf{G}$.

3.2.1 Network Architecture. The proposed network f (shown in Fig. 7) is composed by two major parts: a network for target query feature generation (top), and branches for initial skeleton component prediction (bottom). For the first part, we extend the architecture of Deformable DETR (DEtection TRansformer) [72] for encoding and decoding the target query features. Deformable DETR is a Transformer-based architecture for object detection and has better performance than DETR [9] in both accuracy and computational efficiency, which is suitable to our line and point targets.

Different from the original decoder output for bounding boxes, GCD aims to predict lines, points and their corresponding Graph ID. The proposed decoder also produces auxiliary predictions of Type and IsBranch. For this, we remove the linear projection and the `sigmoid` function in the final layers and keep the target query embedding (namely, Target Query Features) for our prediction tasks (see Fig. 7 (top-right)). In our case, the Target Query Features are composed of two parts, namely Line Query Features and Point Query Features, for line and point targets, respectively. In detail, we now introduce the network branches for the five prediction tasks:
(i) Lines: a 3-layer feed-forward neural network (FFN) is added on top of the Line Query Features as the detection head. The FFN predicts an output of size $N_{line} \times 4$ with each n-th line vector represented in the format of $[[x_1^{(n)}, y_1^{(n)}], [x_2^{(n)}, y_2^{(n)}]]$ with coordinates positions relative to the image width and height.
(ii) isBranch: a linear projection is added on top of the Line Query Features, followed by a `softmax` function for classifying whether a predicted line belongs to $\mathbf{G}$ or not. For each line in $\widehat{\mathbf{L}}$, isBranch gives the scores for two classes (yes and no), e.g. $[0.99, 0.01]$.
(iii) Points: a FFN is added on top of the Point Query Features as the detection head. The FFN predicts an output of size $N_{point} \times 2$ with each n-th point denoted by its relative coordinates in the format of $[x^{(n)}, y^{(n)}]$, e.g. $[0.56, 0.74]$. Similar to Lines, relative coordinates denote the positions relative to the image width and height.
(iv) Type: a linear projection is added on top of the Point Query Features, followed by a `softmax` function for classifying the type of a predicted point. For each point in $\widehat{\mathbf{E}}$ and $\widehat{\mathbf{J}}$, Type gives the scores for three classes ('end', 'junction', 'others'), e.g. $[0.98, 0.01, 0.01]$.
(v) Graph ID: a linear projection is added on top of the Target Query Features for Graph ID estimation. This is motivated by the fact that, in contrast to traditional classification tasks with pre-defined classes, the number of sub-graphs within $\mathbf{G}$ is unknown. For this, we employ the idea of associative embedding [34] to express the output for joint target prediction and grouping. The general idea

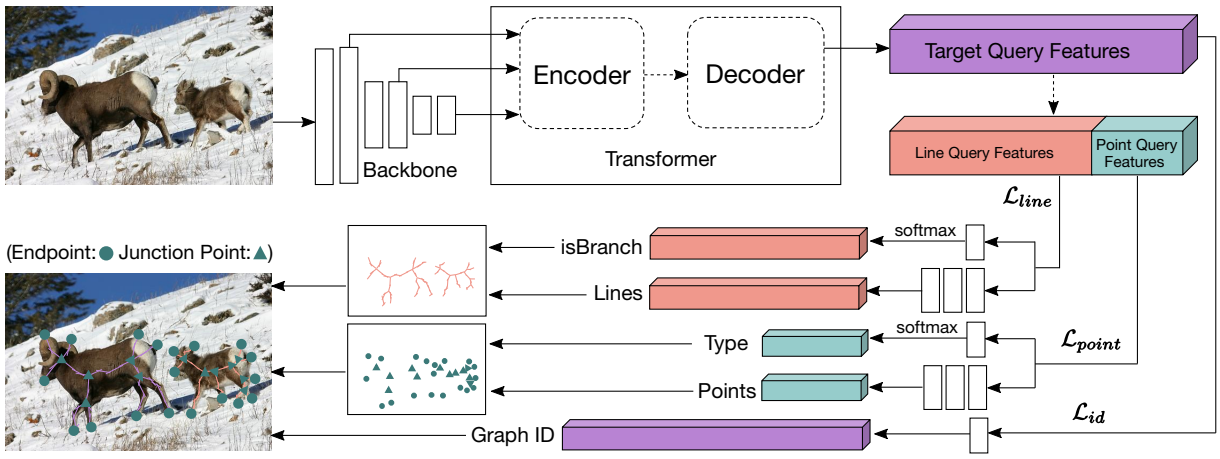

Figure 7: GCD network. We extend the transformer architecture for encoding and decoding skeleton components such as lines and points.

is that for each target prediction (Lines and Points), an embedding serves as a "tag" to identify its sub-graph assignment. All predictions associated with the same tag value belong to the same sub-graph. As the Graph ID is projected from the Target Query Features, it is inherently corresponding to the predicted lines and points.

Outputs from the above branches will be employed for pre-processing and skeleton graph reconstruction in the following Graph Reconstructor. We observe that the decomposition strategy and the Transformer-based model f proposed for the GCD is promising, as visualized in Fig. 7 (bottom-left).

3.2.2 Loss Function. The total loss function $\mathcal{L}$ is composed of three terms that encode the differences between the predicted results $(\widehat{\mathbf{E}}, \widehat{\mathbf{J}}, \widehat{\mathbf{L}})$ and their corresponding ground truths $(\mathbf{E}, \mathbf{J}, \mathbf{L})$:

$$\mathcal{L}(\theta) = \omega_1 \mathcal{L}_{line}(\theta) + \omega_2 \mathcal{L}_{point}(\theta) + \omega_3 \mathcal{L}_{id}(\theta) \quad . \tag{3}$$

where $\mathcal{L}_{line}$ is the line loss, $\mathcal{L}_{point}$ is the point loss, and $\mathcal{L}_{id}$ represents the ID loss.

Line Loss: The line loss $\mathcal{L}_{line}$ measures the difference between the predicted $\widehat{\mathbf{L}}$ (from the GCD branch Lines and isBranch) and the ground truth line $\mathbf{L}$. Specifically, $\mathcal{L}_{line}$ is calculated based on the Hungarian algorithm [22] to find a bipartite matching between the ground truth and predicted lines. While each line is represented as a vector (based on two points), the distance between a pair of line segments $d(\widehat{\mathbf{l}}, \mathbf{l})$ is calculated by the L1 loss for point locations plus the Binary Cross Entropy (BCE) loss for isBranch, $\widehat{\mathbf{l}} \in \widehat{\mathbf{L}}$. We empirically weigh the L1 and BCE losses with coefficients 5 and 1 respectively to balance their numerical values. To ensure the completeness of skeleton branches, we enforce that the number of predictions should be more than the maximum number of ground truth based on our preliminary statistics in Section 3.1. Thus, each ground truth corresponds to at least one prediction. Finally, the loss $\mathcal{L}_{line}$ is computed as the average distance of all matched line pairs:

$$\mathcal{L}_{line} = \frac{1}{K} \sum_{k=1}^{K} d(\widehat{\mathbf{l}}_k, \mathbf{l}_k) \quad . \tag{4}$$

where $(\widehat{\mathbf{l}}_k, \mathbf{l}_k)$ is a matched line pair.

Point Loss: The point loss $\mathcal{L}_{point}$ measures the difference between predicted points $(\widehat{\mathbf{E}}, \widehat{\mathbf{J}})$ (from the GCD branch Points and Type) and the ground truth $(\mathbf{E}, \mathbf{J})$. Here, the endpoints and junction points are all considered as Points and classified by Type (end, junction, and other). Since each point is encoded by relative coordinates and a pre-defined class, the distance between a point pair $d(\widehat{\mathbf{p}}, \mathbf{p})$ is calculated by the L1 loss for point locations plus the Cross Entropy (CE) loss for Type, $\widehat{\mathbf{p}} \in (\widehat{\mathbf{E}}, \widehat{\mathbf{J}})$, $\mathbf{p} \in (\mathbf{E}, \mathbf{J})$. We set the weights for L1 and CE losses to 5 and 1, and utilize the Hungarian algorithm to match the points such that the following term is optimized:

$$\mathcal{L}_{point} = \frac{1}{M} \sum_{m=1}^{M} d(\widehat{\mathbf{p}}_m, \mathbf{p}_m) \quad . \tag{5}$$

where $(\widehat{\mathbf{p}}_m, \mathbf{p}_m)$ is a matched point pair while M is the total number of points in $(\mathbf{E}, \mathbf{J})$.

ID Loss: ID Loss $\mathcal{L}_{id}$ is used to encourage pairs of tags (from the GCD's Graph ID branch) to have similar values if the corresponding predictions belong to the same sub-graph or dissimilar values if otherwise. It should be noted that there are no exact "ground truth" tags for the network to predict, since what matters is not the particular tag values, but only the differences between them. Motivated by this fact, let $g_i \in \mathbb{R}$ be a predicted sub-graph id of a target (point or line) for $1 \le i \le (K+M)$. The reference embedding $\overline{g}_i$ would be the original sub-graph id after normalization by the maximum value (i.e. maximum number of sub-graphs). The ID loss is computed by the squared distance between the reference embedding and the predicted embedding for each target. It also includes a regularization penalty that drops exponentially as the distance between the two tags increases. We model the ID loss $\mathcal{L}_{id}$:

$$\mathcal{L}_{id} = \frac{1}{K+M} \sum_{i=1}^{K+M} (\overline{g}_i - g_i)^2 + \frac{1}{(K+M)^2} \sum_{i=1}^{K+M} \exp\{(\overline{g}_i - g_i)^2\} \, . \tag{6}$$

where the final distribution of g_i is treated as a one dimensional histogram, and the non-parametric histogram segmentation method [11] is employed for tag clustering.

3.2.3 Implementation. By default, we use input image size of 512×512 pixels, with $N_{points} = 128$ and $N_{lines} = 1024$. The Swin-base in [31] is employed as the backbone and the initial network weights θ are initialized by Xavier [14]. We optimized the loss functions using AdamW [32] with a mini-batch size of 1, an initial learning rate of 2e-4, and weight decay of 1e-4. We ran for a total of 200

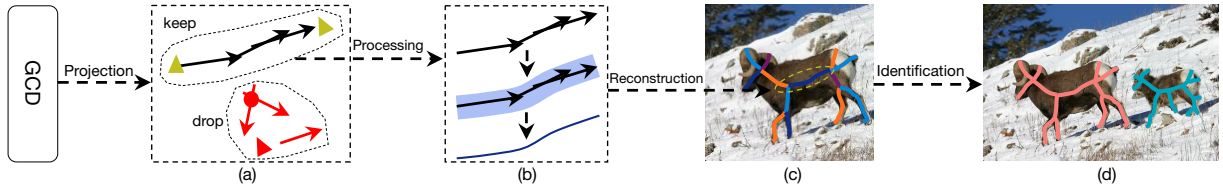

Figure 8: Pipeline of the Graph Reconstructor (GR). Branches in (c) and sub-graphs in (d) are distinguished by colours (orange and cyan).

Algorithm 1: Skeleton_Graph_Reconstruction($\mathbf{I}$, $\widehat{\mathbf{E}}$, $\widehat{\mathbf{J}}$, $\widehat{\mathbf{L}}$)

Data: input image $\mathbf{I}$, prediction $\widehat{\mathbf{E}}$, $\widehat{\mathbf{J}}$, $\widehat{\mathbf{L}}$, distance threshold γ
Result: graph $\widehat{G}$
$\widehat{\mathbf{P}} = \{\widehat{\mathbf{p}} | \ \widehat{\mathbf{p}} \in (NMS(\widehat{\mathbf{E}}), NMS(\widehat{\mathbf{J}}))\ \}$;
$\Delta = \{g \mid g \ is \ the \ predicted \ id \ of \ \widehat{\mathbf{L}} \ and \ \widehat{\mathbf{P}}\}$;
$\widehat{\Delta} = histogram\_segment(\Delta)$;
$\widehat{\mathbf{B}} \leftarrow projection\_processing\_on\_mask(\widehat{\mathbf{L}}, \mathbf{I})$;
$update\_graph\_id(\widehat{\mathbf{B}}, \widehat{\mathbf{P}}, \widehat{\Delta})$;
$N \leftarrow count(\widehat{\mathbf{B}})$, $\widehat{\mathbf{R}} \leftarrow new \ hashmap$;
while $count(\widehat{\boldsymbol{R}}) < 2 * N$ **do**
  $dilate\_one\_pixel(\widehat{\mathbf{P}})$;
  for $(\widehat{\boldsymbol{p}}, \widehat{\boldsymbol{b}})$ *in* $pairs\_checking(\widehat{\boldsymbol{P}}, \widehat{\boldsymbol{B}})$ **do**
    $d = perpendicular\_distance(\widehat{\mathbf{p}}, \widehat{\mathbf{b}})$;
    $(\mathbf{p}'_1, \ \mathbf{p}'_2) \leftarrow two \ ends \ of \ \widehat{\mathbf{b}}$;
    $(d_1, \ d_2) \leftarrow (d + distance(\widehat{\mathbf{p}}, \mathbf{p}'_1), \ d + distance(\widehat{\mathbf{p}}, \mathbf{p}'_2))$;
    if $nearest\_neighbor(\boldsymbol{p}'_1, \widehat{\boldsymbol{p}}, d_1)$ **then**
      $\widehat{\mathbf{R}}\ [\ \mathbf{p}'_1\ ] = \widehat{\mathbf{p}}$;
    end
    if $nearest\_neighbor(\boldsymbol{p}'_2, \widehat{\boldsymbol{p}}, d_2)$ **then**
      $\widehat{\mathbf{R}}\ [\ \mathbf{p}'_2\ ] = \widehat{\mathbf{p}}$;
    end
  end
end
$remove\_unqualified\_pairs(\widehat{\mathbf{B}}, \widehat{\mathbf{P}}, \widehat{\mathbf{R}}, \gamma)$;
$\widehat{G} \leftarrow (\widehat{\mathbf{B}}, \widehat{\mathbf{P}}, \widehat{\mathbf{R}})$;

epochs reducing the learning rate by 10 after 160 epochs. The entire training took 27 hours using a 2080 Ti GPU with the PyTorch library.

3.3 Skeleton Graph Reconstruction

Using the predicted $\widehat{\mathbf{E}}$, $\widehat{\mathbf{J}}$ and $\widehat{\mathbf{L}}$ as input, our proposed GR contains four major steps to output the reconstructed sub-graphs. The pipeline of GR is presented in Fig. 8 with the following specific steps. (1) Projection: We first project the predicted $(\widehat{\mathbf{E}}, \widehat{\mathbf{J}}, \widehat{\mathbf{L}})$ onto a mask of similar size to $\mathbf{I}$. (2) Processing: Points and lines are firstly filtered based on their Type and isBranch and scores respectively. Points with predicted Type 'others' are dropped since they do not correspond to either end points or junction points. Similarly, lines that are predicted as branches (isBranch is true has higher probability) are kept. The kept lines are then processed based on morphological dilation and erosion to generate branch candidates (see (b)). (3) Reconstruction: The point and branch candidates are iteratively merged and connected based on their locations in the mask. Ideally, a point should perfectly overlap with its associated branch, but in practice most of them could be just close to each other (but not exact). For this, the dilation operation is iteratively applied on a point to check for an intersection with a branch. Given a number of mutually exclusive points associated with a branch, points with a larger distance from the branch (or smaller isBranch score) are removed (a process similar to "maximal suppression" of distances). Moreover, branch candidates without any associated points are also discarded in this step. (4) Identification: With the final branch candidates and their Graph ID, sub-graphs of different objects are identified and marked (see (d)). Algorithm 1 presents the reconstruction details.

Discussion. An intuitive understanding of step (3) is that branch and point candidates are mutually projected onto the graph G for a filtering and integration process. However, the two iterative operations within this step bring additional computational complexity to the GR. In practice, step (3) can be further simplified into branch generation procedure based on morphological dilation/erosion and branch growing [41]. In addition to faster speed, our experiments in Section 4.2.3 will show that the end and junction points from the simplified step (3) are already quite promising and are close to the ground truths $\mathbf{E}$ and $\mathbf{J}$. This observation reflects upon the high accuracy and robustness of BlumNet.

4 EXPERIMENTS

We first introduce the datasets used for evaluation. We also assess the proposed BlumNet via a set of ablation studies and comparison experiments. We then verify the usability of BlumNet on shapes. Finally, limitations of the framework are discussed in the last part. We follow the settings in [52] and employ the F-measure score (F-score) to evaluate the skeleton extraction performance.

4.1 Datasets

Images: Four image datasets are employed. SK506 [45] (also known as SK-SMALL) has 506 natural images (300 for training and 206 for testing) and 16 object classes. SK1491 [44] (also known as SK-LARGE) is an extension of the SK506 by selecting more images from the MS COCO dataset [26]. It includes 1,491 images (746 for training and 745 for testing). WH-SYMMAX [42] (or WH in short) contains 328 cropped images (228 for training and 100 for testing). For these aforementioned datasets, there is only one target per image. On the other hand, SymPASCAL [19] (or SP in short) has 1,435 images (648 for training and 787 for testing) with most images containing multiple targets, partial visibility and complex backgrounds.
Shapes: Four widely used datasets are employed for the purpose of evaluating BlumNet on shapes. Animal2000 [5] contains 2,000 shapes (20 categories, 100 shapes each) ranging from poultry and domestic pets to insects and wild animals. MPEG7 [23] contains 1,400 (70 categories, 20 shapes each) shapes with a broad range of challenges with respect to deformation and noises. Kimia216 [40] contains 216 shapes (18 categories, 12 shapes each) selected from the MPEG7 dataset. Tetrapod120 [62] contains 120 tetrapod animal shapes from six classes. For each of these datasets, we randomly select 60% shapes for training while the rest are for testing.

4.2 Ablation Study

Our ablation study mainly focuses on analysing the contributions brought upon by the three parts of BlumNet, namely (1) decomposition strategy, (2) GCD structure and its backbone network, and (3) reconstruction strategy.

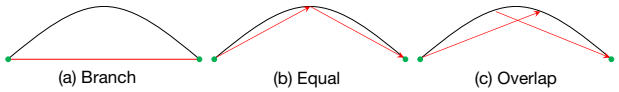

Figure 9: Three different decomposition strategies.

Figure 10: Comparison between extracted skeletons from BlumNet with Equal and Overlap decomposition strategies.

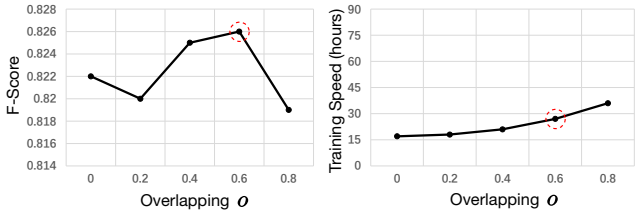

Figure 11: F-score and training speed under different o values.

Table 1: Ablation study on GCD against features extracted from other networks: L-CNN [71], PPGNet [68], AFM [59], HAWP [60], LETR [58] and our method (GCD). Speed: Average inference/image.

	L-CNN	PPGNet	AFM	HAWP	LETR	Ours (GCD)
F-score	0.245	0.114	0.165	0.259	0.724	**0.826**
Speed (secs)	0.085	0.350	0.100	**0.050**	0.069	0.129

4.2.1 Decomposition. Based on the SK1491 datasets, we first compare the proposed decomposition method with other two possible strategies. As presented in Fig. 9, we indicate different methods of decomposing the paths (shown in red), (a) Branch: directly representing a skeleton branch $\mathbf{b}$ using a single line segment $\mathbf{l}$, (b) Equal: representing a skeleton branch $\mathbf{b}$ using a set of connected line segments $\mathbf{l}_k$ without any overlaps, and (c) Overlap: our proposed method, which allows for overlapping line segments. The F-score of the three strategies are: Branch (0.114), Equal (0.822) and Overlap (0.826). Thus, our proposed overlap strategy is the recommended decomposition strategy as it reduces the possibility of branches vanishing in complex structures by detecting multiple, simple, and overlapping line segments (see Fig. 10).

To find a proper degree of overlap o, a set of experiments were conducted based on the SK1491 dataset by varying o from 0 to 0.8. Fig. 11 details the F-score and training speed of BlumNet under different o values. We observe that the F-score reaches its peak (0.826) at $o = 0.6$ and then drops to 0.819 at $o = 0.8$. This is because a higher o means more line segments are generated and hence, not all of them can be properly covered by the fixed number of target query from the decoder. Moreover, since it also requires more time to match additional line segments (at $o = 0.8$), thus we select $o = 0.6$ (marked with red circle) for all the experiments.

4.2.2 GCD. Using the SK1491 dataset, we compare the performance of five existing line segment detectors against our proposed method's GCD. In brief, these are: the CNN-based line detectors include L-CNN [71], PPGNet [68], AFM [59], and HAWP [60], and the recent Transformer-based line detector LETR [58]. In place of GCD, skeleton graph components (both points and lines) are predicted based on the following extracted features: line feature in [71], adjacency matrix in [68], attraction field map in [59], LOI (Line-of-Interest) features in [60], and the fine decoder in [58]. As detailed in Table 1, our proposed GCD achieves the highest F-score with only slightly more inference time required.

Table 2: Ablation study on backbone networks and input sizes.

Backbone	Resnet50	Inceptionv3	Swin-small	Swin-base
Accuracy	0.794	0.804	0.815	**0.826**
Speed (secs)	**0.095**	0.101	0.122	0.129
Input	384x384	448x448	512x512	576x576
Accuracy	0.817	0.818	**0.826**	0.823
Speed (secs)	**0.103**	0.113	0.129	0.143

Table 3: Evaluation of endpoints and junction points from BlumNet. GT: Ground Truth.

	Endpoints		Junction Points	
	Number	Mean Distance	Number	Mean Distance
GT	4471	0	3275	0
BlumNet	4275	16.48	2766	15.07

Table 4: Results on image datasets: SK506, SK1491, WH-SYMMAX (WH), SymPASCAL (SP). Time: Average inference time.

	SK506	SK1491	WH	SP	Time
HED [55]	0.542	0.497	0.732	0.369	**0.010**
SRN [19]	0.632	0.658	0.780	0.443	**0.010**
Hi-Fi [70]	0.681	0.724	0.805	0.454	0.013
DeepFlux [53]	0.695	0.732	0.840	0.502	0.020
AdaLSN [29]	0.740	0.786	0.851	0.497	0.057
BlumNet (VGG16)	0.750	0.792	0.873	0.515	0.090
BlumNet (Swin-base)	**0.752**	**0.826**	**0.877**	**0.521**	0.129

Based on our proposed GCD, we also compare the performance of different backbone networks and input sizes. We find that both Swin-small and Swin-base from [31] have the best performances (0.815 and 0.826) with the latter one slightly higher. Using Swin-base, we investigate various input sizes and finally select 512×512 for the experiments. This is because the inference speed obtained is only marginally slower while it has the highest F-score.

4.2.3 Reconstruction. To evaluate the effectiveness of the Graph Reconstructor, we count the number of end points and junction points, and also calculate their mean distances to the ground truth. As detailed in Table 3, the endpoint number is close to the ground truth (4275 vs. 4471) while less junction points are recalled (2766 vs. 3275). This is because endpoints are close to the object boundary, which typically contain richer color and texture features. The junction points, on the other hand, are mostly located near the centre of regions and highly rely on high-level context features for detection. In terms of the mean distance, we find they are only around 1 pixel difference (16.48 vs. 15.07). In other words, they both have similar accuracy while the recall of endpoints is significantly higher.

4.3 Results on Images

Table 4 presents the comparison of our proposed BlumNet against five CNN-based methods. Since all the other methods use the VGG16 backbone, we present the results of BlumNet with both VGG16 and Swin-base. We observe that BlumNet yields the best performance on all four image datasets. Specifically, it obtained a significantly higher F-score (0.786 vs. 0.826) than the state-of-the-art method AdaLSN

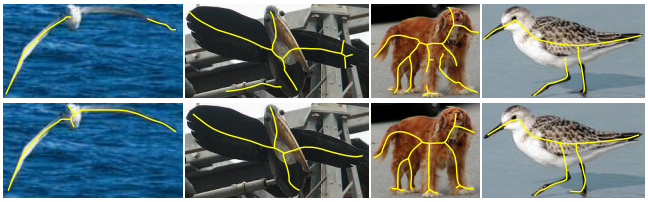

Figure 12: Comparison of extracted skeletons from AdaLSN [29] (top) and our proposed BlumNet (bottom).

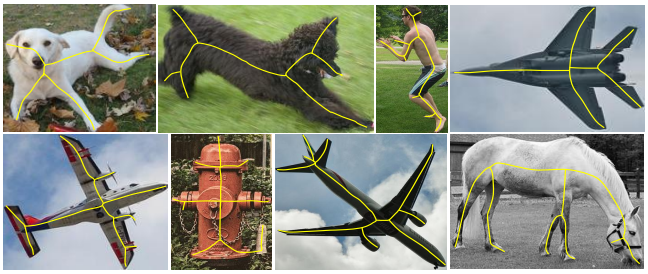

Figure 13: More BlumNet results from selected images of the four image datasets.

while the inference time is slightly longer. Fig. 12 presents the visual comparison between the two methods. It can be observed that skeletons from BlumNet have better quality in terms of connectivity and completeness. More extracted skeletons from BlumNet are shown in Fig. 13: We note that BlumNet skeletons are slim, accurate, mostly complete, and can be directly employed for various tasks including pruning, matching, classification, etc.

Due to limited space, we briefly report the quantitative comparison with other three methods: MSB [56], GeoSkeletonNet [65], and the journal version of DeepFlux [57] (DeepFlux-J). MSB (ResNet-101 backbone by default) achieves 0.754 F1-score. With the same backbone, BlumNet is 0.801. (2) GeoSkeletonNet (VGG16 backbone by default) is 0.757. With the same backbone, BlumNet is 0.792. (3) DeepFlux-J achieves 0.754 and 0.736 with the ResNet-101 and VGG16 backbones, respectively. BlumNet is 0.801 (ResNet-101) and 0.792 (VGG16). In short, under the same setting, BlumNet yields better performance than all three methods, MSB, GeoSkeletonNet, and DeepFlux-J. Concisely, the F1-scores improved by 6.2%, 4.6%, 6.9% (VGG16: 7.6%, ResNet-101: 6.2%), respectively.

In Fig. 3, we visually compare the robustness of BlumNet versus AdaLSN [29] with respect to various deformations. Quantitative comparisons with AdaLSN on the SK1491 dataset shows that the F-scores (of testing set) for AdaLSN and BlumNet are 0.786 and 0.826, respectively (see Table 4). We then randomly rotate, rescale and flip the testing data and this resulted in F-scores of 0.634 (AdaLSN) and 0.820 (BlumNet). This is a strong evidence of BlumNet's robustness against various deformations with only a very marginal 1 point decline while AdaLSN drops almost 15 points.

4.4 Results on Shapes

Table 5 presents the result of BlumNet against state-of-the-art methods on four widely used shape datasets. Similarly, BlumNet also achieves the best performance on these datasets. Compared to images, shapes have simpler backgrounds and skeleton generators should be more focused on geometrical and topological features.

Table 5: Results on shape datasets: Kimia216, Tetrapod120, Animal2000 and MPEG7.

	Kimia216	Tetrapod120	Animal2000	MPEG7
FHN [18]	0.698	0.728	0.706	0.688
U-Net [35]	0.869	0.906	0.893	0.870
DISCO [48]	0.724	0.755	0.745	0.725
SkeNetV2 [33]	0.865	0.901	0.889	0.865
BlumNet	**0.87**	**0.928**	**0.91**	**0.893**

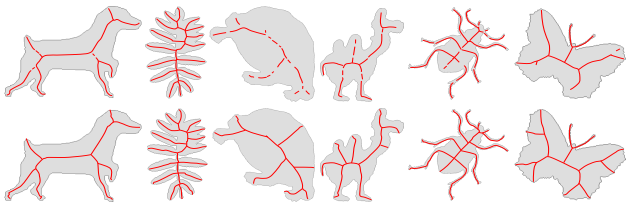

Figure 14: Comparison of extracted skeletons from U-Net [35] (top) and our proposed BlumNet (bottom). Our proposed method is noticeably more complete and accurate.

Our Transformer-based GCD is capable of integrating both local and global features for detecting overlapping line segments, hence skeletons from BlumNet have better quality in terms of connectivity and completeness. Fig. 14 visualizes the extracted skeletons from U-Net [35] versus that of our proposed method. The advantages of BlumNet are most pronounced: slim, low in noise, with promising connectivity and completeness.

4.5 Limitations

We present some future directions for BlumNet based on two specific limitations: the inability to (1) identify overlapped or connected objects, and (2) reconstruct skeletons strictly at the object center. For example (see Fig. 2 (f)), objects in the second image are inadvertently connected. In this case, the Graph ID score in the overlapping region cannot confidently tell apart the two objects. In Fig. 9 (c), since the decomposed line segments do not exactly cover the original branch path, their reconstruction is not strictly at the object center. This is despite our experiments in Table 4 and 5 showing that the BlumNet skeletons are close to the ground truth.

5 CONCLUSION

In this paper, we present a framework, BlumNet, for object skeleton extraction in shapes and images. In contrast to most CNN-based methods that predict skeleton heatmaps, BlumNet directly predicts skeleton graph components via graph decomposition and reconstruction strategies with effective encoding and decoding that are optimized within an extended transformer architecture. Our comprehensive experiments show promising results, achieving state-of-the-art performance on a wide range of image and shape datasets. For reproducibility, the source code of BlumNet can be found at https://github.com/cong-yang/BlumNet.

ACKNOWLEDGMENTS

Research activities leading to this work have been supported by the Scientific Research Fund from Soochow University.

REFERENCES

[1] Ioannis Andreadis, Maria I Vardavoulia, Gerasimos Louverdis, and Nikolaos Papamarkos. 2000. Colour image skeletonisation. In *European Signal Processing Conference*. 1–4.

[2] Carlo Arcelli and G Sanniti Di Baja. 1989. A one-pass two-operation process to detect the skeletal pixels on the 4-distance transform. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 11, 4 (1989), 411–414.

[3] X. Bai and L.J. Latecki. 2008. Path Similarity Skeleton Graph Matching. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 30, 7 (2008), 1282–1292.

[4] X. Bai, L.J. Latecki, and W. Liu. 2007. Skeleton Pruning by Contour Partitioning with Discrete Curve Evolution. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 29, 3 (2007), 449–462.

[5] Xiang Bai, Wenyu Liu, and Zhuowen Tu. 2009. Integrating contour and skeleton for shape classification. In *IEEE International Conference on Computer Vision Workshops*. 360–367.

[6] Harry Blum. 1967. A transformation for extracting new descriptors of shape. *Models for Perception of Speech and Visual Forms* (1967), 362–380.

[7] Jonathan W Brandt and V Ralph Algazi. 1992. Continuous skeleton computation by Voronoi diagram. *CVGIP: Image understanding* 55, 3 (1992), 329–338.

[8] Alexander Bucksch. 2014. A practical introduction to skeletons for the plant sciences. *Applications in plant sciences* 2, 8 (2014), 1400005.

[9] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-end object detection with transformers. In *European conference on computer vision*. 213–229.

[10] W. Choi, K. Lam, and W. Siu. 2003. Extraction of the Euclidean skeleton based on a connectivity criterion. *Pattern Recognition* 36, 3 (2003), 721–729.

[11] Julie Delon, Agns Desolneux, Jos-Luis Lisani, and Ana Beln Petro. 2006. A nonparametric approach for histogram segmentation. *IEEE Transactions on Image Processing* 16, 1 (2006), 253–261.

[12] Michal Drozdzal, Eugene Vorontsov, Gabriel Chartrand, Samuel Kadoury, and Chris Pal. 2016. The importance of skip connections in biomedical image segmentation. In *Deep learning and data labeling for medical applications*. 179–187.

[13] Yaorong Ge and J Michael Fitzpatrick. 1996. On the generation of skeletons from discrete Euclidean distance maps. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 18, 11 (1996), 1055–1066.

[14] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics*. 249–256.

[15] D. Ilke et al. 2019. SkelNetOn 2019: Dataset and Challenge on Deep Learning for Geometric Shape Understanding. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops*. 1–9.

[16] Jeong-Hun Jang and Ki-Sang Hong. 2001. A pseudo-distance map for the segmentation-free skeletonization of gray-scale images. In *IEEE International Conference on Computer Vision*, Vol. 2. 18–23.

[17] Koteswar Rao Jerripothula, Jianfei Cai, Jiangbo Lu, and Junsong Yuan. 2021. Image co-skeletonization via co-segmentation. *IEEE Transactions on Image Processing* 30 (2021), 2784–2797.

[18] Nan Jiang, Yifei Zhang, Dezhao Luo, Chang Liu, Yu Zhou, and Zhenjun Han. 2019. Feature Hourglass Network for Skeleton Detection. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops*. 1–5.

[19] Wei Ke, Jie Chen, Jianbin Jiao, Guoying Zhao, and Qixiang Ye. 2017. SRN: Side-output residual network for object symmetry detection in the wild. In *IEEE Conference on Computer Vision and Pattern Recognition*. 1068–1076.

[20] Ron Kimmel, Doron Shaked, Nahum Kiryati, and Alfred M. Bruckstein. 1995. Skeletonization via Distance Maps and Level Sets. *Computer Vision and Image Understanding* 62, 3 (1995), 382–391.

[21] Debbie Honghee Ko, Ammar Ul Hassan, Saima Majeed, and Jaeyoung Choi. 2021. Skelgan: A font image skeletonization method. *Journal of Information Processing Systems* 17, 1 (2021), 1–13.

[22] H. W. Kuhn. 1955. The Hungarian Method for the assignment problem. *Naval Research Logistics Quarterly* 2 (1955), 83–97.

[23] L. J. Latecki, R. Lakamper, and T. Eckhardt. 2000. Shape descriptors for non-rigid shapes with a single closed contour. In *IEEE Conference on Computer Vision and Pattern Recognition*. 424–429.

[24] Alex Levinshtein, Cristian Sminchisescu, and Sven Dickinson. 2013. Multiscale symmetric part detection and grouping. *International journal of computer vision* 104, 2 (2013), 117–134.

[25] Yibo Li and Han Qu. 2018. LSD and Skeleton Extraction Combined with Farmland Ridge Detection. In *International Conference on Intelligent and Interactive Systems and Applications*. 446–453.

[26] Tsung-Yi Lin et al. 2014. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision*. 740–755.

[27] Tony Lindeberg. 1998. Edge detection and ridge detection with automatic scale selection. *International Journal of Computer Vision* 30, 2 (1998), 117–156.

[28] Chang Liu, Wei Ke, Fei Qin, and Qixiang Ye. 2018. Linear span network for object skeleton detection. In *European Conference on Computer Vision*. 133–148.

[29] Chang Liu, Yunjie Tian, Zhiwen Chen, Jianbin Jiao, and Qixiang Ye. 2021. Adaptive linear span network for object skeleton detection. *IEEE Transactions on Image Processing* 30 (2021), 5096–5108.

[30] Xiaolong Liu, Pengyuan Lyu, Xiang Bai, and Ming-Ming Cheng. 2017. Fusing image and segmentation cues for skeleton extraction in the wild. In *IEEE International Conference on Computer Vision Workshops*. 1744–1748.

[31] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *IEEE International Conference on Computer Vision*. 10012–10022.

[32] Ilya Loshchilov and Frank Hutter. 2018. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*. 1–19.

[33] Sabari Nathan and Priya Kansal. 2021. SkeletonNetV2: A Dense Channel Attention Blocks for Skeleton Extraction. In *IEEE International Conference on Computer Vision Workshops*. 2142–2149.

[34] Alejandro Newell, Zhiao Huang, and Jia Deng. 2017. Associative embedding: end-to-end learning for joint detection and grouping. In *International Conference on Neural Information Processing Systems*. 2274–2284.

[35] Nam Hoang Nguyen. 2021. U-Net Based Skeletonization and Bag of Tricks. In *IEEE International Conference on Computer Vision Workshops*. 2105–2109.

[36] Olivier Petit, Nicolas Thome, Clement Rambour, Loic Themyr, Toby Collins, and Luc Soler. 2021. U-net transformer: Self and cross attention for medical image segmentation. In *International Workshop on Machine Learning in Medical Imaging*. 267–276.

[37] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*. 234–241.

[38] Punam K Saha, Gunilla Borgefors, and Gabriella Sanniti di Baja. 2016. A survey on skeletonization algorithms and their applications. *Pattern recognition letters* 76 (2016), 3–12.

[39] Kacper Sarnacki and Khalid Saeed. 2018. Character Recognition Based on Skeleton Analysis. In *International Conference on Computer Vision and Graphics*. 148–159.

[40] T. B. Sebastian, P.N. Klein, and B.B. Kimia. 2004. Recognition of shapes by editing their shock graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26, 5 (2004), 550–571.

[41] W. Shen, X. Bai, R. Hu, H. Wang, and L. J. Latecki. 2011. Skeleton growing and pruning with bending potential ratio. *Pattern Recognition* 44, 2 (2011), 196–209.

[42] Wei Shen, Xiang Bai, Zihao Hu, and Zhijiang Zhang. 2016. Multiple instance subspace learning via partial random projection tree for local reflection symmetry in natural images. *Pattern Recognition* 52 (2016), 306–316.

[43] Wei Shen, Xiang Bai, XingWei Yang, and Longin Jan Latecki. 2013. Skeleton pruning as trade-off between skeleton simplicity and reconstruction error. *Science China Information Sciences* 56, 4 (2013), 1–14.

[44] Wei Shen, Kai Zhao, Yuan Jiang, Yan Wang, Xiang Bai, and Alan Yuille. 2017. Deepskeleton: Learning multi-task scale-associated deep side outputs for object skeleton extraction in natural images. *IEEE Transactions on Image Processing* 26, 11 (2017), 5298–5311.

[45] Wei Shen, Kai Zhao, Yuan Jiang, Yan Wang, Zhijiang Zhang, and Xiang Bai. 2016. Object skeleton extraction in natural images by fusing scale-associated deep side outputs. In *IEEE Conference on Computer Vision and Pattern Recognition*. 222–230.

[46] Tom Sie Ho Lee, Sanja Fidler, and Sven Dickinson. 2013. Detecting curved symmetric parts using a deformable disc model. In *IEEE international Conference on Computer Vision*. 1753–1760.

[47] Amos Sironi, Vincent Lepetit, and Pascal Fua. 2014. Multiscale centerline detection by learning a scale-space distance transform. In *IEEE Conference on Computer Vision and Pattern Recognition*. 2697–2704.

[48] Soonyong Song, Heechul Bae, and Junhee Park. 2021. DISCO - U-Net Based Autoencoder Architecture With Dual Input Streams for Skeleton Image Drawing. In *IEEE International Conference on Computer Vision Workshops*. 2128–2135.

[49] Xiaojun Tang, Rui Zheng, and Yinghao Wang. 2021. Distance and Edge Transform for Skeleton Extraction. In *IEEE International Conference on Computer Vision Workshops*. 2136–2141.

[50] Yudi Tang, Lei He, Wei Lu, Xin Huang, Hai Wei, and Huaiguang Xiao. 2021. A novel approach for fracture skeleton extraction from rock surface images. *International Journal of Rock Mechanics and Mining Sciences* 142 (2021), 104732.

[51] Stavros Tsogkas and Sven Dickinson. 2017. Amat: Medial axis transform for natural images. In *IEEE International Conference on Computer Vision*. 2708–2717.

[52] Stavros Tsogkas and Iasonas Kokkinos. 2012. Learning-based symmetry detection in natural images. In *European Conference on Computer Vision*. 41–54.

[53] Yukang Wang, Yongchao Xu, Stavros Tsogkas, Xiang Bai, Sven Dickinson, and Kaleem Siddiqi. 2019. Deepflux for skeletons in the wild. In *IEEE Conference on Computer Vision and Pattern Recognition*. 5287–5296.

[54] Shih-En Wei, Varun Ramakrishna, Takeo Kanade, and Yaser Sheikh. 2016. Convolutional pose machines. In *IEEE conference on Computer Vision and Pattern Recognition*. 4724–4732.

[55] Saining Xie and Zhuowen Tu. 2015. Holistically-nested edge detection. In *IEEE International Conference on Computer Vision*. 1395–1403.

[56] Weijian Xu, Gaurav Parmar, and Zhuowen Tu. 2019. Geometry-Aware End-to-End Skeleton Detection. In *British Machine Vision Conference*, Vol. 2. 7.
[57] Yongchao Xu, Yukang Wang, Stavros Tsogkas, Jianqiang Wan, Xiang Bai, Sven Dickinson, and Kaleem Siddiqi. 2021. Deepflux for skeleton detection in the wild. *International Journal of Computer Vision* 129, 4 (2021), 1323–1339.
[58] Yifan Xu, Weijian Xu, David Cheung, and Zhuowen Tu. 2021. Line segment detection using transformers without edges. In *IEEE Conference on Computer Vision and Pattern Recognition*. 4257–4266.
[59] Nan Xue, Song Bai, Fudong Wang, Gui-Song Xia, Tianfu Wu, and Liangpei Zhang. 2019. Learning attraction field representation for robust line segment detection. In *IEEE Conference on Computer Vision and Pattern Recognition*. 1595–1603.
[60] Nan Xue, Tianfu Wu, Song Bai, Fudong Wang, Gui-Song Xia, Liangpei Zhang, and Philip HS Torr. 2020. Holistically-attracted wireframe parsing. In *IEEE Conference on Computer Vision and Pattern Recognition*. 2788–2797.
[61] Cong Yang, Bipin Indurkhya, John See, and Marcin Grzegorzek. 2020. Towards Automatic Skeleton Extraction with Skeleton Grafting. *IEEE Transactions on Visualization and Computer Graphics* (2020), 1–1.
[62] Cong Yang, Oliver Tiebe, Kimiaki Shirahama, and Marcin Grzegorzek. 2016. Object Matching with Hierarchical Skeletons. *Pattern Recognition* 55 (2016), 183–197.
[63] Cong Yang, Oliver Tiebe, Kimiaki Shirahama, Ewa Łukasik, and Marcin Grzegorzek. 2017. Evaluating contour segment descriptors. *Machine Vision and Applications* 28, 3 (2017), 373–391.
[64] Cong Yang, Wenfeng Wang, Yunhui Zhang, Zhikai Zhang, Lina Shen, Yipeng Li, and John See. 2021. MLife: a lite framework for machine learning lifecycle initialization. *Machine Learning* 110, 11 (2021), 2993–3013.
[65] Fan Yang, Xin Li, and Jianbing Shen. 2020. MSB-FCN: multi-scale bidirectional FCN for object skeleton extraction. *IEEE Transactions on Image Processing* 30 (2020), 2301–2312.
[66] Qiaoping Zhang and Isabelle Couloigner. 2007. Accurate centerline detection and line width estimation of thick lines using the radon transform. *IEEE Transactions on Image Processing* 16, 2 (2007), 310–316.
[67] T. Y. Zhang and C. Y. Suen. 1984. A Fast Parallel Algorithm for Thinning Digital Patterns. *Commun. ACM* 27, 3 (1984), 236–239.
[68] Ziheng Zhang, Zhengxin Li, Ning Bi, Jia Zheng, Jinlei Wang, Kun Huang, Weixin Luo, Yanyu Xu, and Shenghua Gao. 2019. Ppgnet: Learning point-pair graph for line segment detection. In *IEEE Conference on Computer Vision and Pattern Recognition*. 7105–7114.
[69] Zheng Zhang, Wei Shen, Cong Yao, and Xiang Bai. 2015. Symmetry-based text line detection in natural scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2558–2567.
[70] Kai Zhao, Wei Shen, Shanghua Gao, Dandan Li, and Ming-Ming Cheng. 2018. Hi-Fi: hierarchical feature integration for skeleton detection. In *International Joint Conference on Artificial Intelligence*. 1191–1197.
[71] Yichao Zhou, Haozhi Qi, and Yi Ma. 2019. End-to-end wireframe parsing. In *IEEE International Conference on Computer Vision*. 962–971.
[72] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. 2021. Deformable DETR: Deformable Transformers for End-to-End Object Detection. In *International Conference on Learning Representations*. 1–16.